Practical 5

Aim: Implement Error Recovery in the Syntax Analyzer: Extend Experiment 4 by incorporating error recovery techniques to detect and report syntax errors in the input program.

Prac5.l:

```
%{
#include "y.tab.h"
#include<stdlib.h>
extern int yylval;
%}

%%
[0-9]+     {yylval=atoi(yytext); return NUMBER;}
[ \t]      ;
\n         {return '\n';}
[+*()]     {return yytext[0];}
%%

int yywrap()
{
return 1;
}
```

Prac5.y:

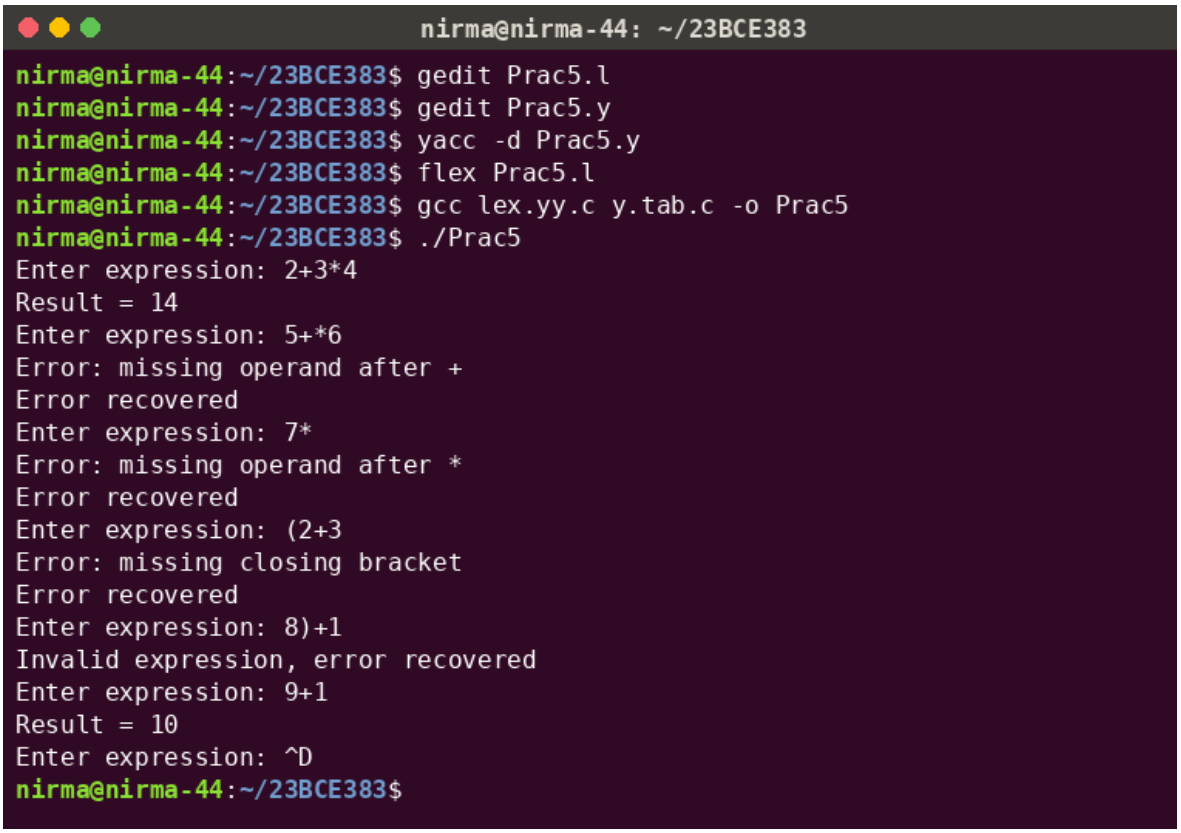
```
%{
#include<stdio.h>
#include<stdlib.h>
int yylex();
void yyerror(char *s);
int err=0;
%}

%token NUMBER
%left '+'
%left '*'
%%
input: input line
     | line
     ;
line: expr '\n' {
        if(err==0) printf("Result = %d\n",$1);
        else printf("Error recovered\n");
        err=0;
        printf("Enter expression: ");
      }
    | error '\n' {yyerrok; printf("Invalid expression, error recovered\nEnter expression
    ;
expr: expr '+' expr {$$=$1+$3;}
    | expr '*' expr {$$=$1*$3;}
    | '(' expr ')' {$$=$2;}
    | NUMBER {$$=$1;}
    | expr '+' error {yyerrok; printf("Error: missing operand after +\n"); err=1;}
    | expr '*' error {yyerrok; printf("Error: missing operand after *\n"); err=1;}
    | '(' expr error {yyerrok; printf("Error: missing closing bracket\n"); err=1;}
    ;
%%
void yyerror(char *s)
```

```
{
}

int main()
{
printf("Enter expression: ");
yyparse();
return 0;
}
```

Output:



Conclusion:

In this practical different syntax errors like missing operand and missing bracket were detected using error productions with the error token. Using yyerrok the parser recovered and continued with the next input.